

Test Coverage – Course and Tutor Registration Feature

Document Title	Test Coverage – Course and Tutor Registration Feature
Application	MindBridge AI Tutor
Prepared By	Kirill Kirilin
Date	21 September 2026
Version	1.0
Feature Scope	Multi-step course and tutor registration workflow, including session browsing, cart management, and mock payment checkout.

1. Test Coverage Table

Test Type	Description	Environment	Environment Justification	Pass/Fail Criteria
Functional Testing	Verifies that the end-to-end registration workflow completes correctly: browsing and filtering sessions, opening a course detail page, adding a course to the cart, and completing a mock checkout with valid payment details.	QA (localhost:8081)	Functional testing requires a fully deployed, running application and a populated database, but it does not require real student PII. The QA environment's scrubbed database supplies realistic but anonymized test data, so the full workflow can be exercised repeatedly without any risk to real accounts or payment systems. Running these tests in STAGING would unnecessarily expose realistic PII, and DEV is too unstable for reliable end-to-end results.	PASS: A student can browse, add a course to the cart, and complete mock checkout; the order total matches the course price (\$164.39) and a confirmation is returned. FAIL: Any step cannot be completed, or the cart total does not match the course price.
Negative / Error Handling Testing	Verifies how the checkout handles invalid or unexpected input in the Card Number, Expiration Date, CVV, and Postal Code fields — including the known defect where a special character (for example, @) bypasses front-	QA (localhost:8081), with DEV (localhost:3000) as a secondary environment for developer debugging	Negative testing must be run against a complete, deployed application because the defect occurs when the front end passes unvalidated input to the server, which only happens in a full stack. QA provides that stack with anonymized payment data, so deliberately malformed input and server errors carry no risk to real users or financial systems. DEV is listed as a secondary environment because	PASS: All invalid input (special characters, empty fields, over-length values, non-numeric card numbers) is rejected by validation with a clear, field-level error message and an HTTP 4xx response. FAIL: Any invalid input reaches the server unvalidated,

Test Type	Description	Environment	Environment Justification	Pass/Fail Criteria
	end validation and returns an HTTP 500 error.		developers can attach logs and a debugger there once QA has confirmed the reproduction steps; it is not the primary environment because it is unstable and rarely used by QA directly.	returns an HTTP 500, or produces an unhandled error.
Regression Testing	Verifies that after a bug fix, code change, or new build, the previously working registration workflow and its related areas (search, filters, cart totals, navigation) still behave as expected.	QA (localhost:8081)	Regression testing depends on a stable, controlled baseline so that results before and after a change are directly comparable; the QA environment provides that consistency along with a repeatable scrubbed data set. Running regression suites in DEV would produce unreliable results because the code there changes continuously, and using STAGING would consume the final validation environment with routine checks and expose realistic PII for no benefit.	PASS: All previously passing test cases still pass, the reported defect is confirmed fixed, and no new defects appear in related areas. FAIL: Any previously passing case now fails, or the fix introduces a new defect.
Usability Testing	Assesses whether a real student can move through the multi-step registration process intuitively — finding a session, understanding pricing and course details, adding it to the cart, and completing checkout without confusion or assistance.	STAGING (localhost:8082)	Usability testing measures how the product feels to a real user, so it must be run against production-like content, performance, and data volume; anonymized placeholder records in QA would not reflect what a student actually sees. STAGING is the only environment with an unscrubbed database containing realistic simulated student names and ages, which makes the experience representative while the data itself remains simulated rather than real. Because participants may see this realistic PII, sessions must use test accounts and follow the team's data-handling rules.	PASS: Participants complete registration unaided within the expected time, with no misinterpreted labels or navigation errors, and rate the flow as clear. FAIL: Participants need assistance, abandon the flow, or repeatedly misunderstand a step or field.
Responsive Design Testing	Verifies that the registration workflow displays and functions correctly across screen sizes and devices, including the known defect where the primary navigation menu overlaps the main content below a 400px	QA (localhost:8081)	Responsive testing needs a stable, deployed build whose layout can be examined repeatedly at fixed breakpoints using DevTools device emulation; QA provides that stability without requiring real PII, since layout behaviour is independent of the data shown. Testing in DEV would give inconsistent results as styles change between builds, and STAGING should be reserved for final validation	PASS: At 320px, 375px, 400px, 768px, and 1280px widths, all content and controls remain visible, readable, and clickable, with no overlap between the navigation menu and the main content. FAIL: Any element overlaps, is cut off, or cannot be

Test Type	Description	Environment	Environment Justification	Pass/Fail Criteria
	viewport width.		once the breakpoints already pass in QA.	interacted with at any tested width.

2. Known Defects Affecting This Feature

The following defects were identified by the QA team lead before test execution began. They are documented here as pre-existing, known issues, which is distinct from defects discovered during execution.

Defect ID	DEF-001
Description	Entering a special character (for example, @) into the payment field during checkout bypasses front-end validation and causes the application to return an HTTP 500 Internal Server Error.
Visible In	Browser Network tab (failed request returning status 500); no user-facing error message is displayed.
Recommended Test Environment	QA (localhost:8081)
Reason	This defect can be safely reproduced and documented in the QA environment using anonymized test payment data. Testing in QA eliminates any risk to real user accounts or financial systems while allowing developers to investigate the issue.

Defect ID	DEF-002
Description	When the browser viewport is reduced below 400px in width (mobile emulation), the primary navigation menu overlaps the main content area, making it impossible to interact with the page.
Visible In	User interface — the browser viewport under mobile emulation, using the DevTools device toolbar.
Recommended Test Environment	QA (localhost:8081)
Reason	This is a front-end layout defect that depends only on viewport width, so it can be reproduced reliably in QA by resizing the viewport below 400px with DevTools device emulation. QA is stable and isolated, so the overlap appears identically on every run and no real users or data are affected. Developers can also confirm the fix there before the build is promoted to STAGING.